

* gradient at node = incoming gradient $\times$ local derivative
 //_

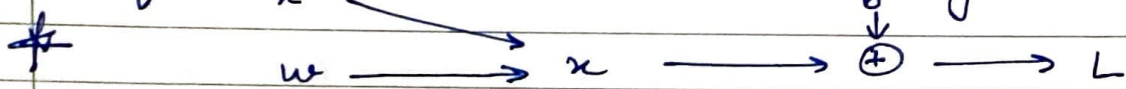
9th Feb 2026 / DAG2101

→ Mechanism of Autograd

- DAG (dynamic directed acyclic graph)

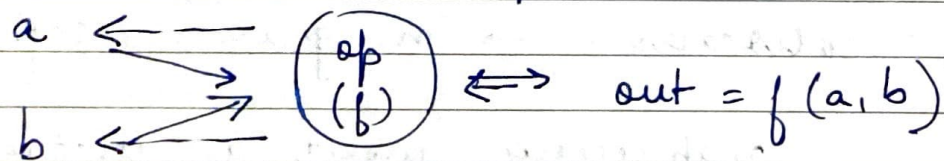
Tensors (Nodes) $\rightarrow$ store the data & accumulated gradient

funcⁿ obj. (edges) $\rightarrow$ every operaⁿ that creates grad that knows its opⁿ leaf tensors $\rightarrow$ tensors created by the user.



- Autonomy of the graph node
 In pyTorch every operator is a node that implements the forward & backward method.

Forward: compute out



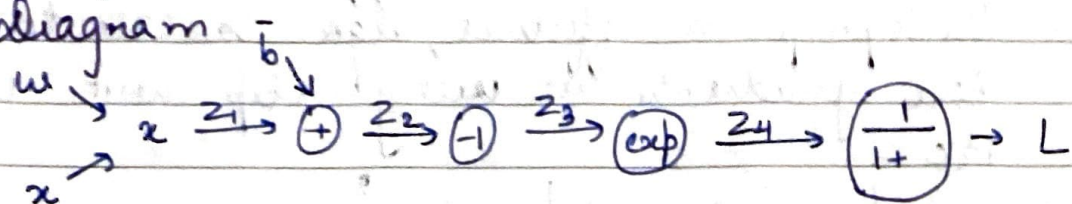
backward $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial out} \cdot \frac{\partial out}{\partial a}$

- Eg. $L = \sigma(w \cdot x + b)$

$L = \frac{1}{1 + e^{-(w^T x + b)}}$

- | | |
|----------------------------|---------------|
| 1. $z_1 = w \cdot x$ | mul. |
| 2. $z_2 = z_1 + b$ | add. |
| 3. $z_3 = -z_2$ | neg. |
| 4. $z_4 = \exp(z_3)$ | exp. |
| 5. $L = \frac{1}{1 + z_4}$ | inv. +
add |

diagram



We want to find out $\frac{\partial L}{\partial w}$, $\frac{\partial L}{\partial b}$.

- We now need to move backwards node-by-node. The wt. gradient $\partial L / \partial w$ is the product of all local derivatives encountered on the path from L back to w.

Key is $\rightarrow$ local grad. + so far grad.

Table inserts here -

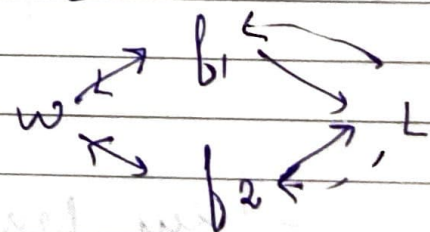
$$L = \frac{1}{1+2y} \quad \frac{\partial L}{\partial 2y} = -\frac{1}{(1+2y)^2} \quad -L^2$$

$$z_4 = \exp(z_3) \quad \frac{\partial z_4}{\partial z_3} = \exp(z_3) \quad -L^2 z_4$$

$$z_3 = -z_2 \quad \frac{\partial z_3}{\partial z_2} = (-1) \cdot L^2 z_4 \quad L^2 z_4$$

- Why only graphs? Gradient accumulaⁿ

If a var. w affects L thru 2 diff. paths, the gradients are summed at the joinⁿ



$$1 \rightarrow \frac{\partial L}{\partial p_1}$$

$$2 \rightarrow \frac{\partial L}{\partial p_2}$$

$$\text{Total } \frac{\partial L}{\partial w} = \frac{\partial L}{\partial p_1} + \frac{\partial L}{\partial p_2}$$

fan-out method

2 efficient

- Backprop. is effective algo. to calculate the the gradients at every step w/ variables, etc

eg. $L = (w \cdot x + b - y)^2$

$z_1 = w \cdot x$

mul

$z_2 = z_1 + b$

add

$z_3 = z_2 - y$

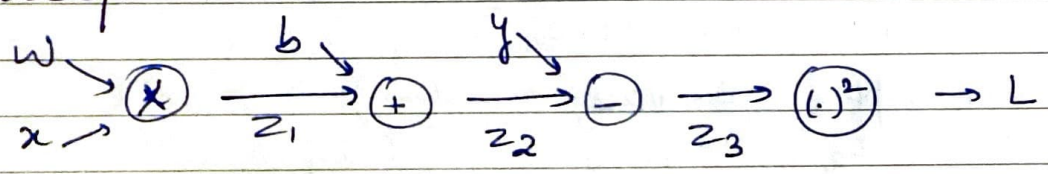
neg.

$L = z_3^2$

power $\rightarrow$ square / mul

Flow backward pass $\rightarrow$ msg. passing

diag. enters here



step-by-step process

- at power node $(z_3^2) \Rightarrow z_3$, $1 \cdot z_3$
- at subtraction $(z_2 - y) \Rightarrow 1$, $z_3 \times 1$
- at add $(z_1 + b) \Rightarrow 1$, 1
- at multiplication $(w \cdot x) \Rightarrow x$, $z_3 \cdot x$

- key features

- $\rightarrow$ efficiency $\rightarrow$ only local derivatives at each node
- $\rightarrow$ intermediate results are added / cached
- $\rightarrow$ grad. accumulaⁿ $\rightarrow$ diff. paths are summed
- $\rightarrow$ PyTorch handles branching in the same way.

//_

→ FP (Forward Pass)

- data flows from $L \rightarrow R$
- each op $\leadsto$ saves intermediate values
- each tensor pts. to a grad. fn

→ BP (Backward Pass)

- graph is traversed in reverse.
- local grads. are calculated using saved values.
- grads. are multiplied by the incoming gradient (chain rule)

- The graph logic →

- define tensors (requires_grad = True)
- perform operations
- call backward()
- access grad attribute
 $\hookrightarrow (\partial L / \partial w)$

- eg. tiny network for hand calculaⁿ

$$2 \rightarrow x_1 = 0.5, x_2 = 1.0$$

2 hidden neurons

1 output neuron

diag.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$y = 1 \text{ (true label)}$$

$$u_{11} = 0.5$$

$$u_{12} = 0.5$$

$$u_{21} = 0.5$$

$$u_{22} = 0.5$$

$$w_1 = 1.0$$

$$w_2 = 1.0$$

(Forward Pass)

$$z_1 = u_{11}x_1 + u_{21}x_2 + b = 0.75$$

$$h_1 = \sigma(z_1) = 0.679$$

$$z_2 = u_{12}x_1 + u_{22}x_2 + b = 0.75$$

$$h_2 = \sigma(z_2) = 0.679$$

$$z_{wt} = w_1h_1 + w_2h_2 + b = 1.358$$

$$\hat{y} = \sigma(z_{wt}) = 0.796$$

Loss (BCE)

$$\begin{aligned} \text{Error} &= \hat{y} - y \\ &= 0.796 - 1 \\ &= -0.204 \end{aligned}$$

$$L = - (y \log(y) + (1-y) \log(1-\hat{y}))$$

$$= - \log(0.796) = 0.228$$

$$\delta_{out} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -0.204$$

↓
we predicted too low
(should increase output)

(Backward Pass) → calculate gradients
wrt w_1 & w_2

Feb 18th → Quiz

Assignment 1 →
Deadline